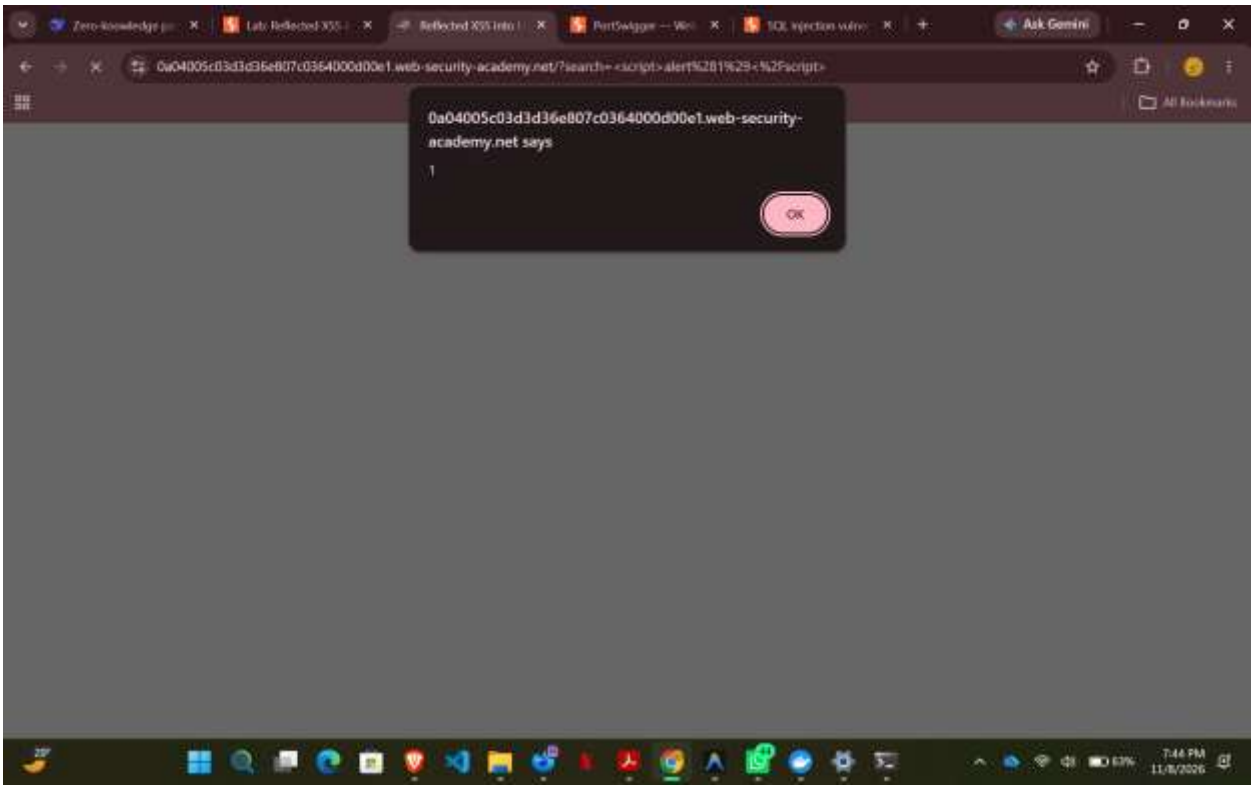
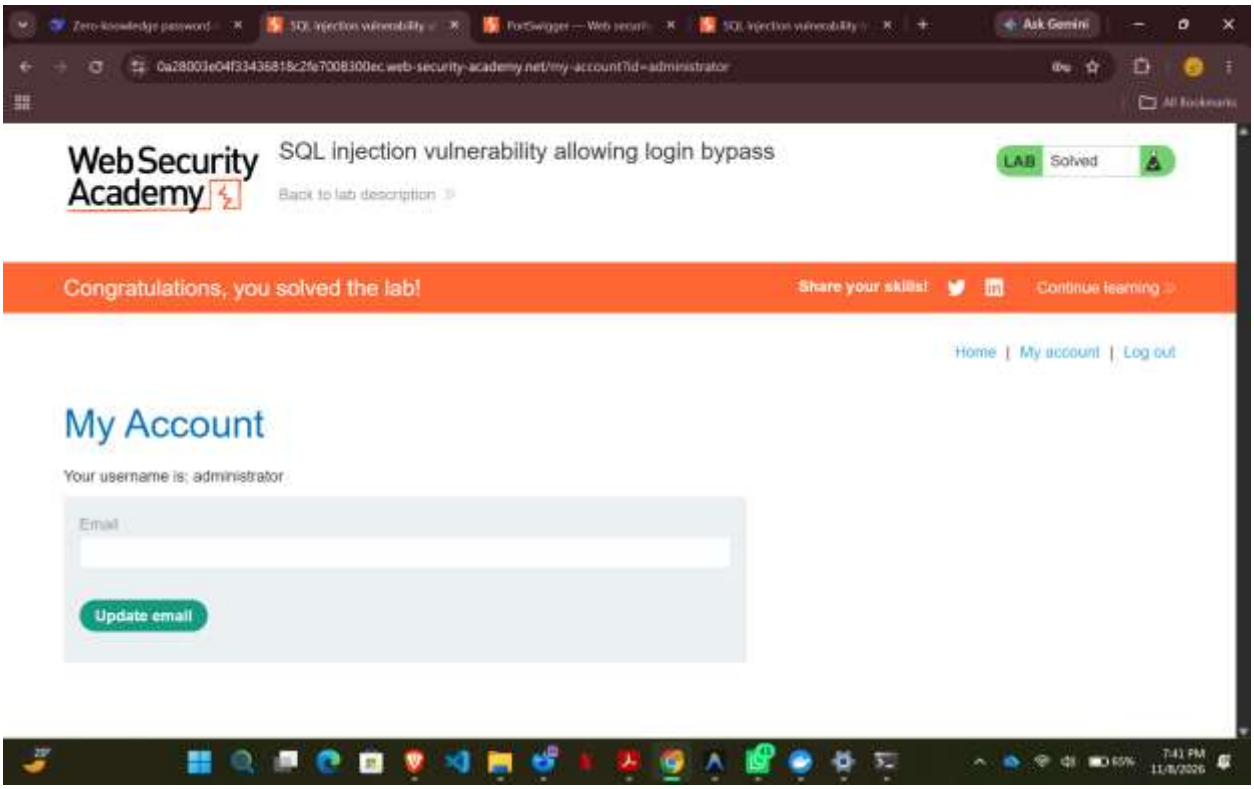
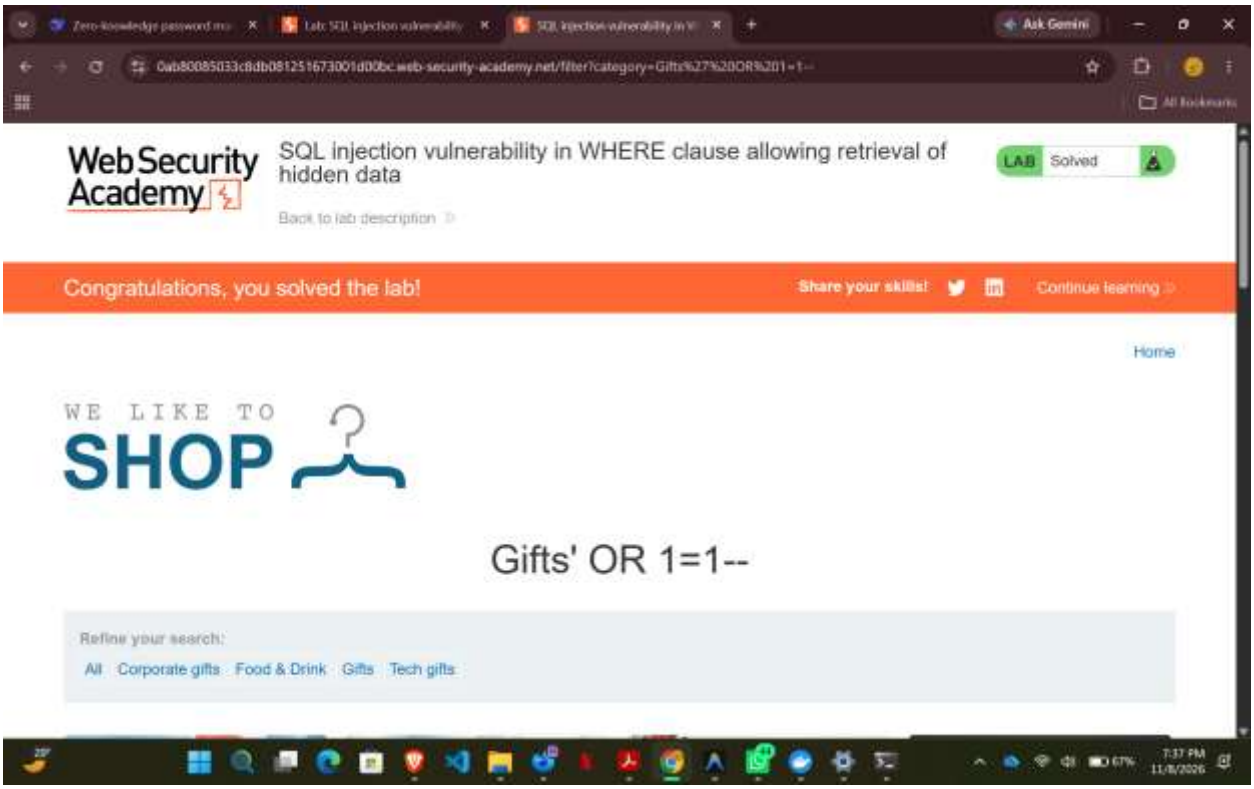


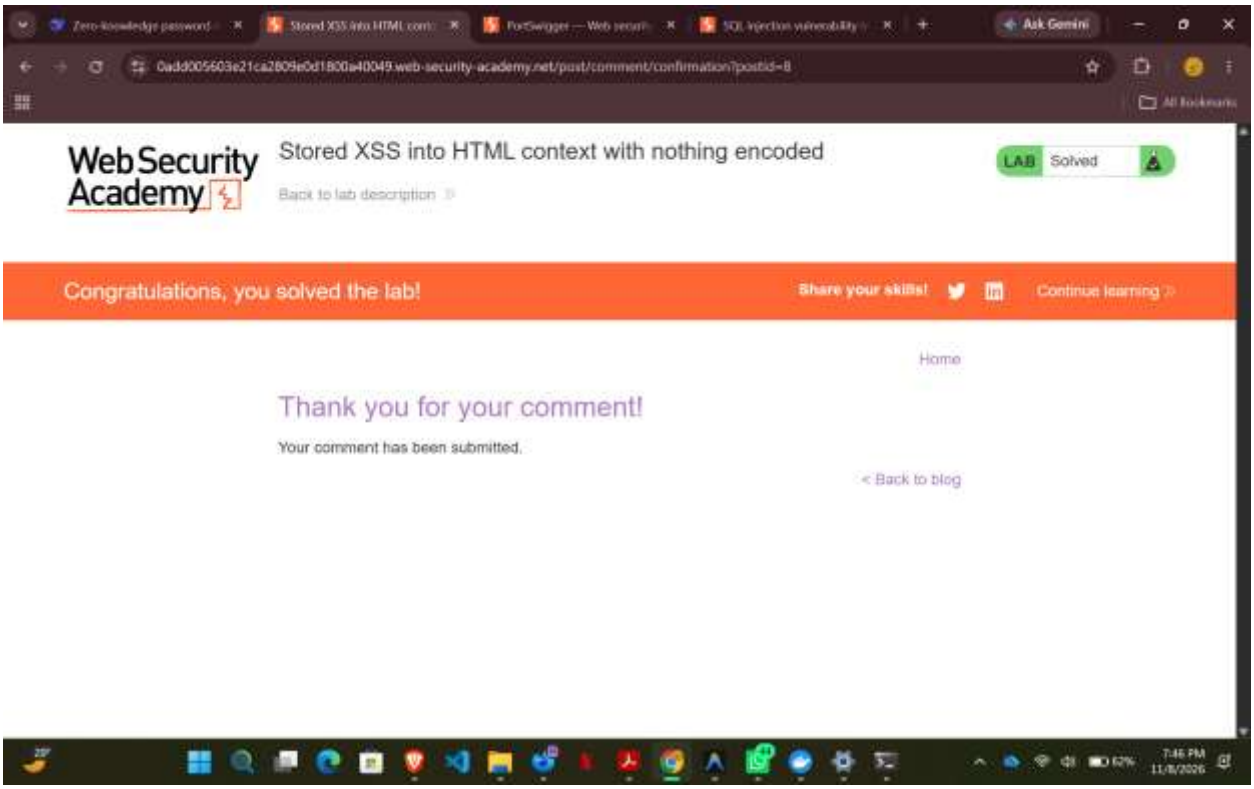
- **Payload Used:** <script>alert(1)</script>
- **Vulnerability Explanation:** The application takes a search query from the URL and reflects it directly into the HTML response without any encoding. The browser parses the <script> tag as executable JavaScript, causing the alert to pop up. This is "reflected" because the payload is immediately returned in the HTTP response and is not stored.
- **Defensive Measure:** Apply **output encoding** (e.g., HTML entity encoding) to all user-supplied data before rendering it in the browser. Set the Content-Type header to text/html; charset=utf-8 and use a strict Content Security Policy (CSP).



- **Payload Used:** Username: administrator'--
- **Vulnerability Explanation:** Similar to the first exercise, the login form concatenates input into a query like SELECT * FROM users WHERE username = '...' AND password = '...'. By using administrator'--, the query becomes SELECT * FROM users WHERE username = 'administrator'--' AND password = '...'. The database ignores the password check because it has been commented out, logging us in as the admin without a password.
- **Defensive Measure:** Use **parameterized queries** for authentication. Additionally, use generic error messages (e.g., "Invalid username or password") to avoid revealing database details.



- **Payload Used:** ' OR 1=1-- appended to the category parameter.
- **Vulnerability Explanation:** The application takes user input and concatenates it directly into an SQL query without sanitization. By injecting OR 1=1, we force the WHERE clause to evaluate to true for every record. The -- comments out the rest of the original query (which was likely filtering for released = 1), causing the database to return all hidden and unreleased products.
- **Defensive Measure:** Use **parameterized queries** (prepared statements). This ensures the database treats user input as data, not executable code.



- **Payload Used:** <script>alert(1)</script> (posted as a blog comment)
- **Vulnerability Explanation:** The application stores user input (a comment) in its database without sanitization. When the page is loaded again, the script is rendered in the browser and executes for anyone who visits the page. This is "stored" XSS and is more dangerous because it affects other users.
- **Defensive Measure:** Never trust user input. Use **HTML entity encoding** on the server side before saving to the database, and apply **output encoding** when rendering the data back to the user. Implement a strong Content Security Policy (CSP).